



Essay / Assignment Title:

Eco-Travel Advisor – Conversational Agent for Sustainable Tourism

Planning using the Rasa Platform

Programme title:

**MSc Artificial Intelligence - Module: Advanced Conversational UI
Design and Chatbot Development**

Name:

Mohammadreza Gheyshivand

Year:

2026 October

CONTENTS

CONTENTS	2
INTRODUCTION	4
TASK 1: CONDUCTING IN-DEPTH RESEARCH	6
TASK 2: ANALYSIS OF FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS	11
TASK 3: CREATION OF CONVERSATIONAL PROTOTYPE AND USER INTERACTION DESIGN.....	14
TASK 4: PROGRAMMING WORK, AI, AND NLP TECHNIQUES	16
TASK 5: TESTING	20
TASK 6: DEPLOYMENT	25
CONCLUSION	26
REFERENCES	27
APPENDIX	29

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters):

Mohammadreza Gheyshivand

Date: 10/04/2026

Introduction

The tourism and transporting is a big source of climate change these days. In 2019 it caused about 8.8% of global greenhouse gas emissions(Sun et al., 2024). Flying with plane is also very unequal. Only 11% of people who did fly in 2018, and the most frequent flyers, at most 1% of those people, probably cause more than half of passenger flight emissions(Gössling and Humpe, 2020).

Travel platforms now show carbon figures aswell. Flight emissions has appeared in over 65 billion searches (Travalyst, 2024). Researchers also see a role for chatbots that help tourists keep sustainability habits(Majid, Tussyadiah and Kim, 2025).

This report describe a Eco-Travel Advisor(I named it MGH Travel Advisor), a chatbot that help people plan lower-carbon trips as the assignment request. The bot is going to asks short questions about the destination, route, dates, budget and sustainability goals. Then it compare the travels and options by carbon, price and the user's preference, and shows each result in green, amber or red. It can also check the weather, convert currency and hand the chat over to a human advisor pretty easily.

I didn't use a large language model but I used the Rasa NLU and its requirements to make the bot able understanding the meaning of user and generating the answer. In a 2024 benchmark, GPT-4 solved only 0.6% of 1,225 travel planning tasks (Xie et al., 2024). As I said, I used Rasa Open Source with the DIET model, which has been trained much faster than fine-tuning BERT (Bunk et al., 2020).

The bot use free public data, without saved data as a backup, after each turn off and turn on or clearance the conversation (which we have a button for it) any information will be foundable in the application. It is a prototype: booking is a mock, and hotel prices and train fares are estimates. Some tools has been named in the brief, such as the ClimaTiq and Amadeus APIs, are not used. Further we say why.

The code is on GitHub: [Link](#).

The report follows the six tasks of the brief and will ends with a conclusion.

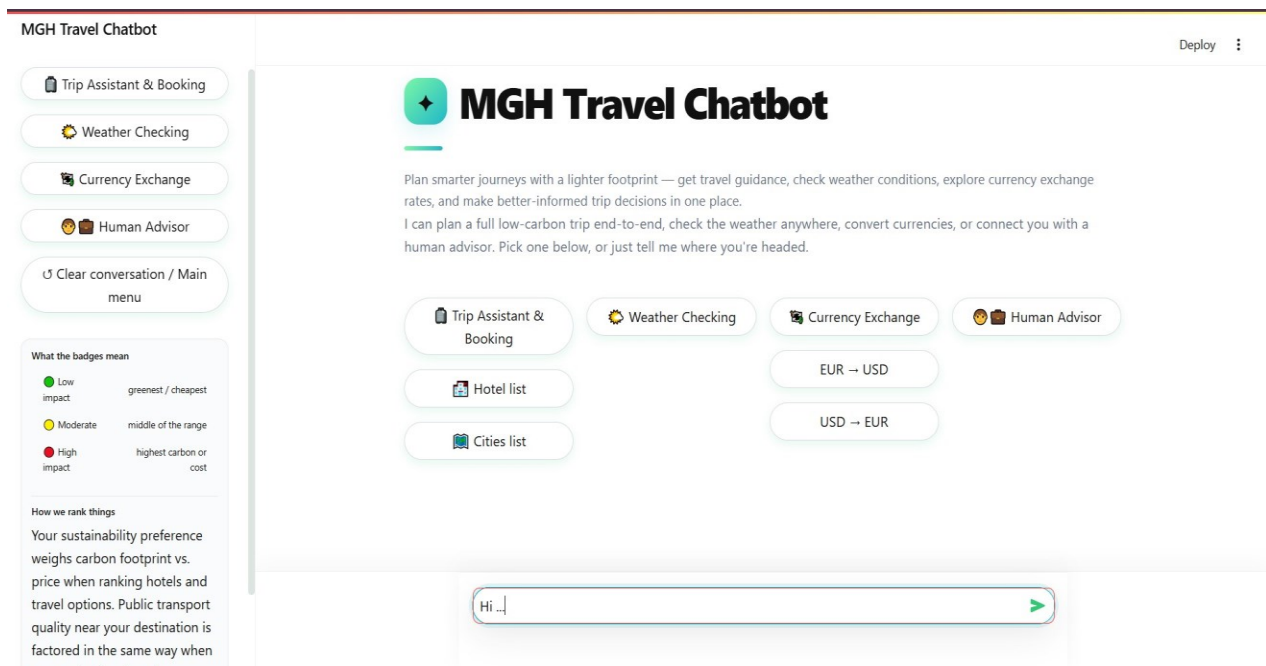


Figure 1

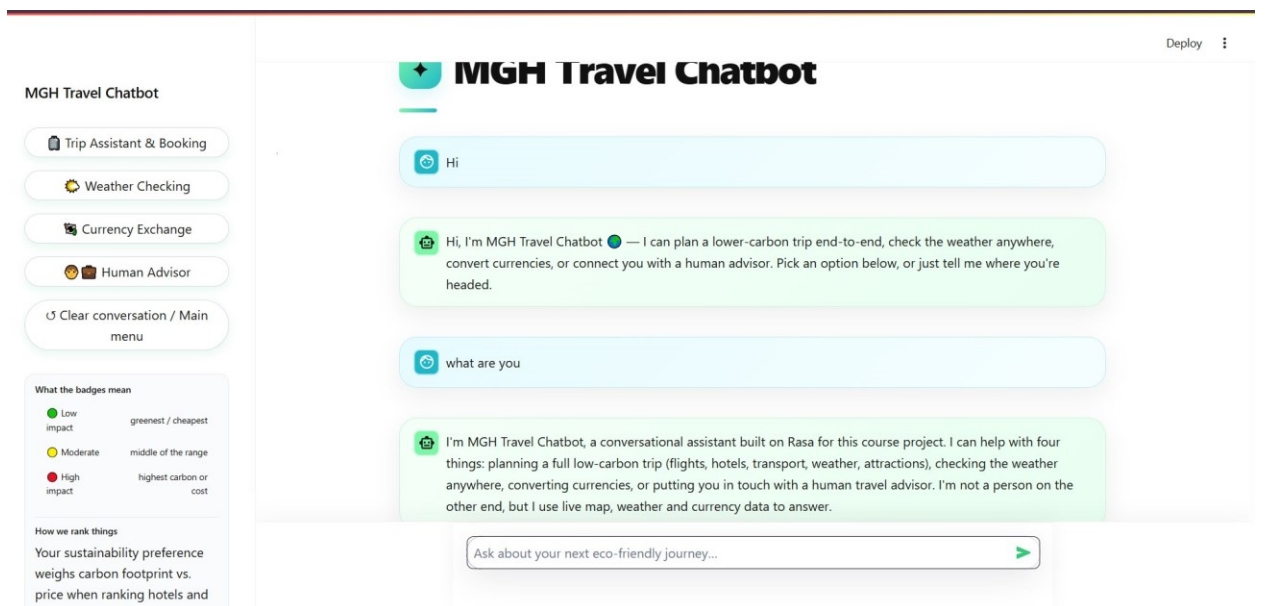


Figure 2

Task 1: Conducting In-Depth Research

I have recently reviewed some research papers and one industry report from 2020 or later. And this is a short review, not a systemic one review, and it has been shaped the design of the bot.

Sustainable travel chatbots

This Industry already are putting lots of carbon data into researches. Travel Impact Model data was shown in over 65 billion searches on Google, Booking.com, Skyscanner and many other platforms, so all of the users can compare flights by emissions aswell as the price (Travalyst, 2024). In researches, one study built a theoretical model of how one chatbot could help tourists keep sustainable transport habits after his trip. It did find that privacy concerns and access to transport affects adoption (Majid, Tussyadiah and Kim, 2025).

NLP, multimodal input and LLMs

Large language model look really attractive for trip planning bots, but they are not reliable yet. In the Travel Planner benchmarks, the model GPT-4 solved only 0.6% of 1,225 planning tasks, it could be a shame for that.(Xie et al., 2024). A lighter option is DIET, which have intents and entities in one model and doesn't need any large pre-trained model or Large language.(Bunk et al., 2020).

So obviously my bot uses DIET and fixed rules, not an LLM. The brief allows a DistilBERT pipeline also, but I kept the lighter DIET. The bot is taking text and button clicks only for now. None of my sources test voice or image input for travel, so this stays an open gap.

User experience and persuasive design

In a questionnaire study, users liked efficient helping and disliked bots that misunderstand them or repeat the same question (Følstad and Brandtzaeg, 2020).

I used this in two different and the same ways at the same time. The bot is asking a short question at a time, and after a misunderstandings in user command it keep asking about the purpose and don't forget the slot for the user's answer. It also shows each option in green, amber or red colors next to the price, to nudge users towards low-carbon choices. Whether this changes real choices still needs testing. Task 5 reports the whole user tests.

Ethics

Privacy concerns affect whether tourists use a travel chatbots like this one.(Majid, Tussyadiah and Kim, 2025). Guidances on accessible chatbots and voice assistants covers many needings, such as cognitive and sensory needings, but it is still not conclusive (Lister et al., 2020).

The brief named greenwashing as a risk which is expressed in the assignment pdf. So now the bot is showing prices and fares as estimates and says that booking is a mock. It keeps the chat in memory and has no database for user data but after each clearance or restarting it will erase the data also. Its colour labels also carry text and a some new icons. It works in English language only.

Gaps, opportunities and risks

The industry report covers flights only, and the chatbot study is theory. In these sources I found no tool that for comparing transport, hotels and weather with a user's budget in just one chat. This bot aims to fill that gap: one short conversation that ranks a whole trip by carbon by kg, price by euro and personal preference as anything that user may have, and will pass the hard cases to a human. Tourism emissions grew 3.5% a year from 2009 to 2019, but technology improved by only 0.3% in one year (Sun et al., 2024).

The main risks are wrong estimates, no voice input (just for now), no screen-reader test, English only and a very small review.

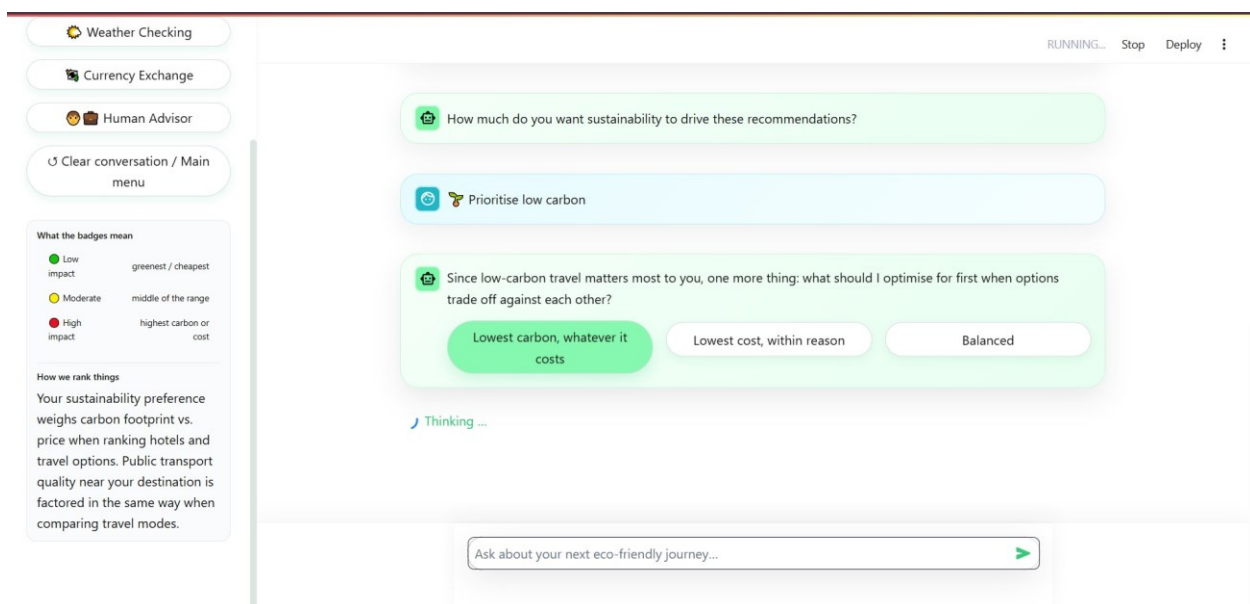


Figure 3

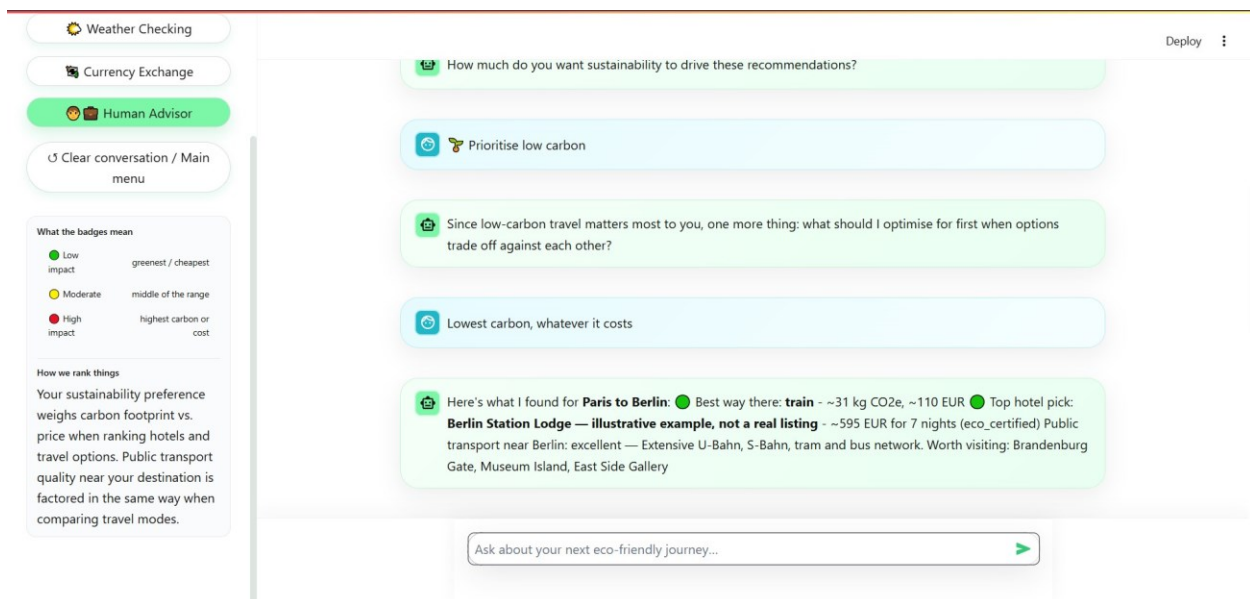


Figure 4

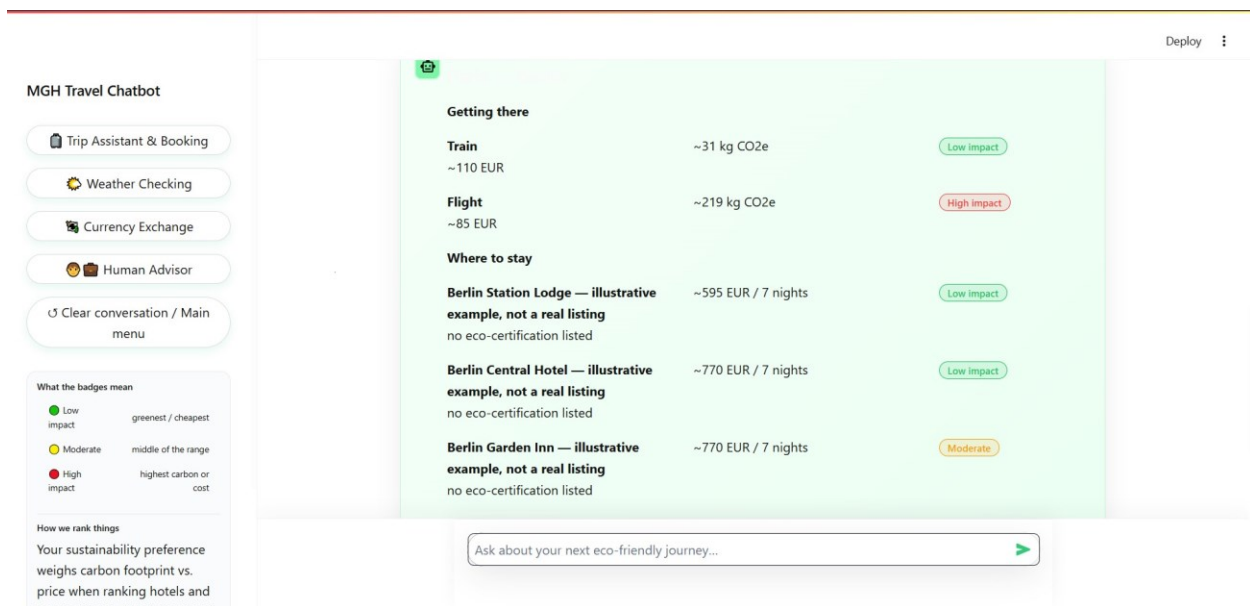


Figure 5

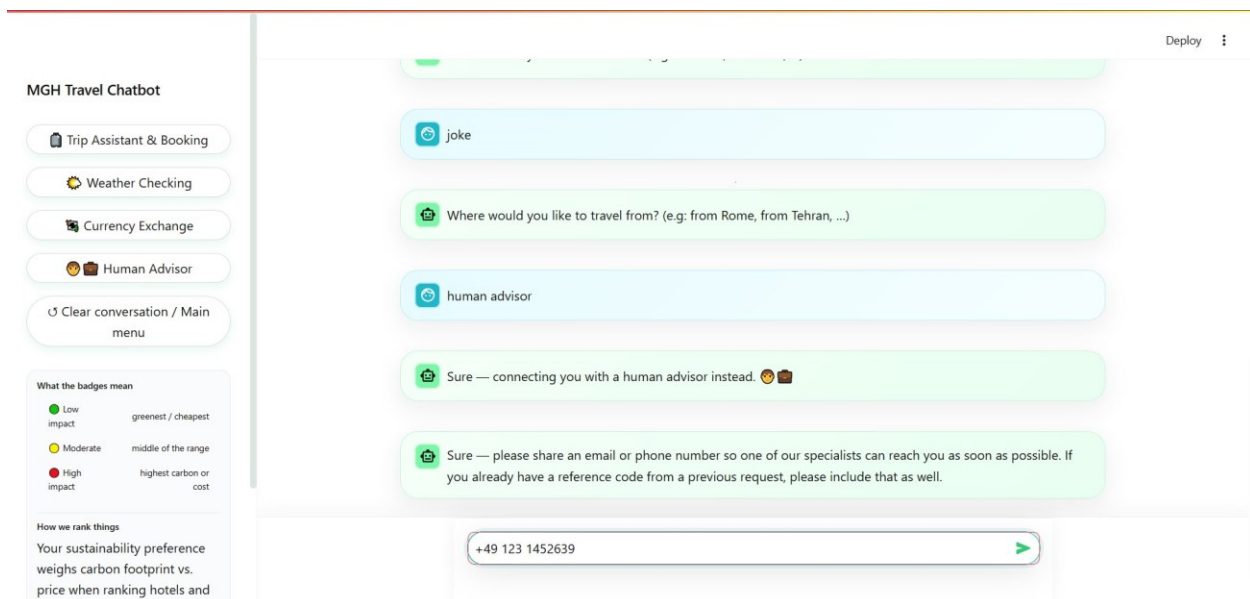


Figure 6

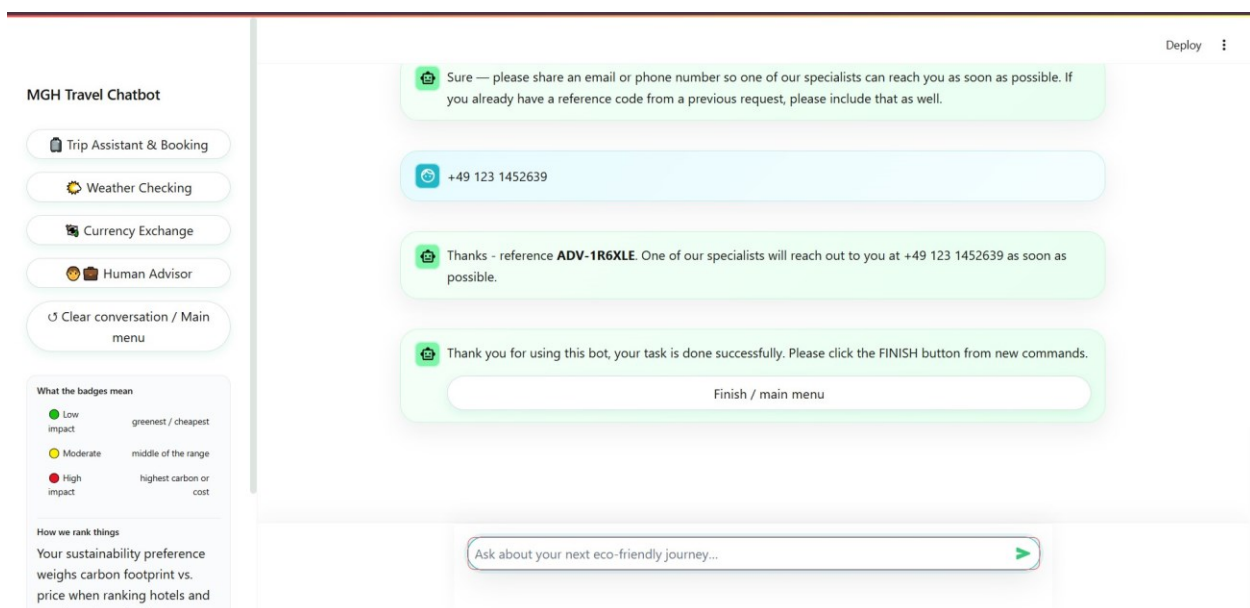


Figure 7

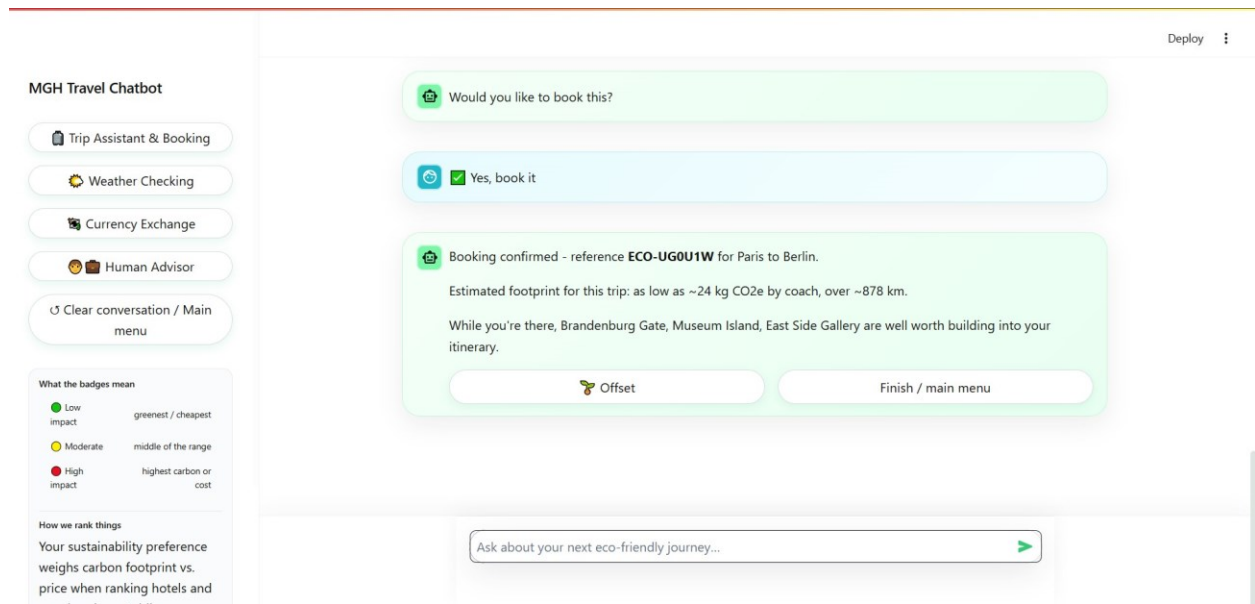


Figure 8

Task 2: Analysis of Functional and Non-Functional Requirements

I've checked each requirements in the brief: what in the bot meets it, and what it is missing.

Functional requirements

Requirement	How the bot meets it	Limit
Trip intakes	A Rasa form ask to eight questions, one after each other. As I developed and you can check, the last one is asked only for the highest sustainability level.	English only and the Budget will be always in euro.
Locations	The user type or tap a city from button. Coordinates come from Nominatim or a saved list of about more than 170 biggest cities.	No GPS, which would add permission prompt plus more data.
Verified informations	A data service try live sources first (OpenStreetMap, Open-Meteo, Wikipedia), then saved data from apis. A hotel card says eco-certified only if OpenStreetMap has a Green Key or ecolabel tag. Carbon use a local table (I explained more at the Task 4).	Station counts, not timtables. Landmarks, not events. Tags will be volunteer-made.
Human escalation	The handover action pack the filled slots generally, the last 20 chats lines and the last intents, and give a reference code.	Printed to console just.
Error recovery	A custom fallback system ask the same questions until the user write the correct answer and it never forget the slots and question. Misspelt cities get a "Did you mean" question.	Small towns may be missed.

Non-functional requirements

Requirement	How the bot meets it	Limit
Usability	Just one short question per turn, with buttons also. A sidebar legend is explaining the all of the colours and labels.	No tooltips.
Reliability	Every browser session have the own chat for itself. Saved data will answer if a live API failed.	Not load tested. Chats will be lost when Rasa get restart (Rasa, n.d.).
Latency	The trip card call several APIs at the same time. A live call can wait up to 15 second, with one retry (it will be 2 time at the end), before saved data answer the bot.	Three seconds is not guaranteed.
Accessibility	Dark text on a light page. Label use words, not only the colour.	The text and buttons just, no voice input.
Data privacy	No database. Chats stay in memory, and only place users names, coordinates and currency codes go to outside of any services.	No privacy notice yet.

The users dislike chatbots that misunderstand them or repeat questions more and more (Følstad and Brandtzaeg, 2020). Colours must not be the only way to show meaning (World Wide Web Consortium, 2024), and guidance on accessibility of chatbot is not yet conclusive for now (Lister et al., 2020). The good design alone don't prove GDPR compliance, that limits the personal data to what is necessary and keep it no longer than those really needed (European Union, 2016).

the briefs' scope also is asking about context-aware suggestions. After weather checking or hotel listing, a user can say "I want to go there" or something like that to really go to that location, and my bot keep that city as the real destination. user can also answer four questions in one message. The formula is: intent travel + from travel_date_from + to travel_date_to + from origin_city + to destination_city.

For example: "I want to go to a travel from this weekend to next weekend from London to Paris". This will fill destination_city, origin_city, travel_date_from and travel_date_to, so the bot will be skipping questions number 1 to 4 and will ask how many people are travelling and user should answer with number or say just me (or something like that also).

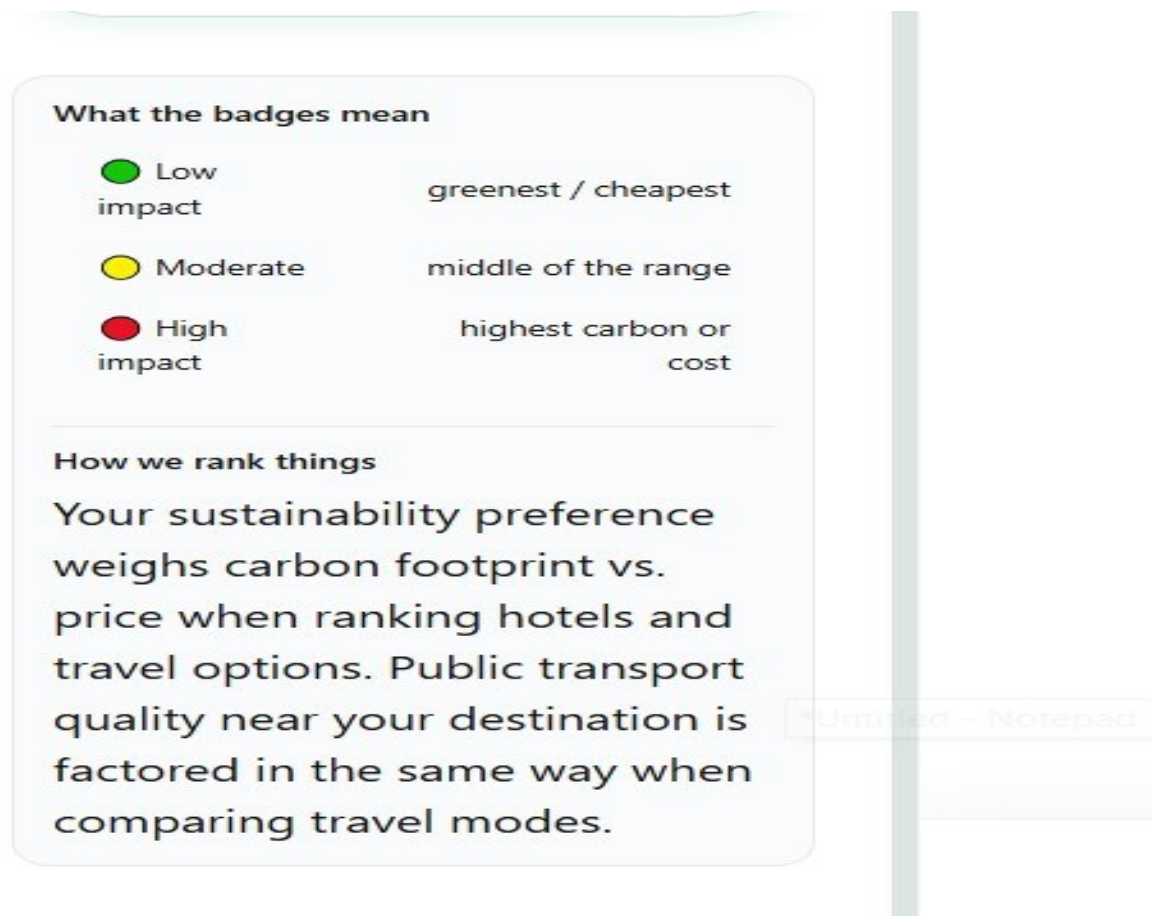


Figure 9

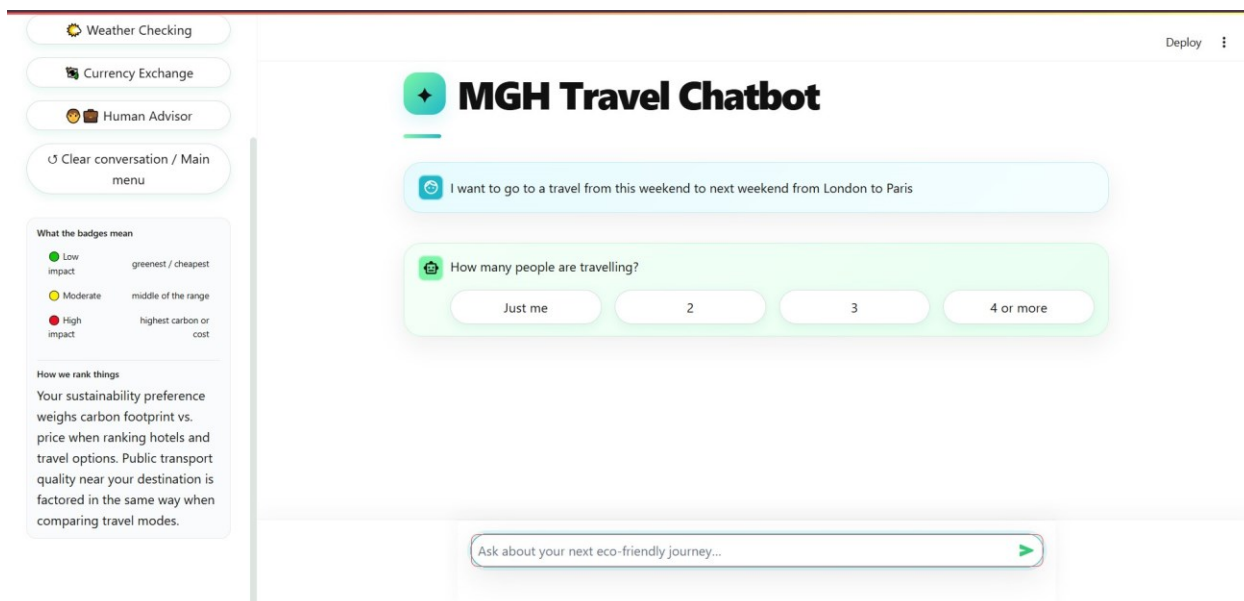


Figure 10

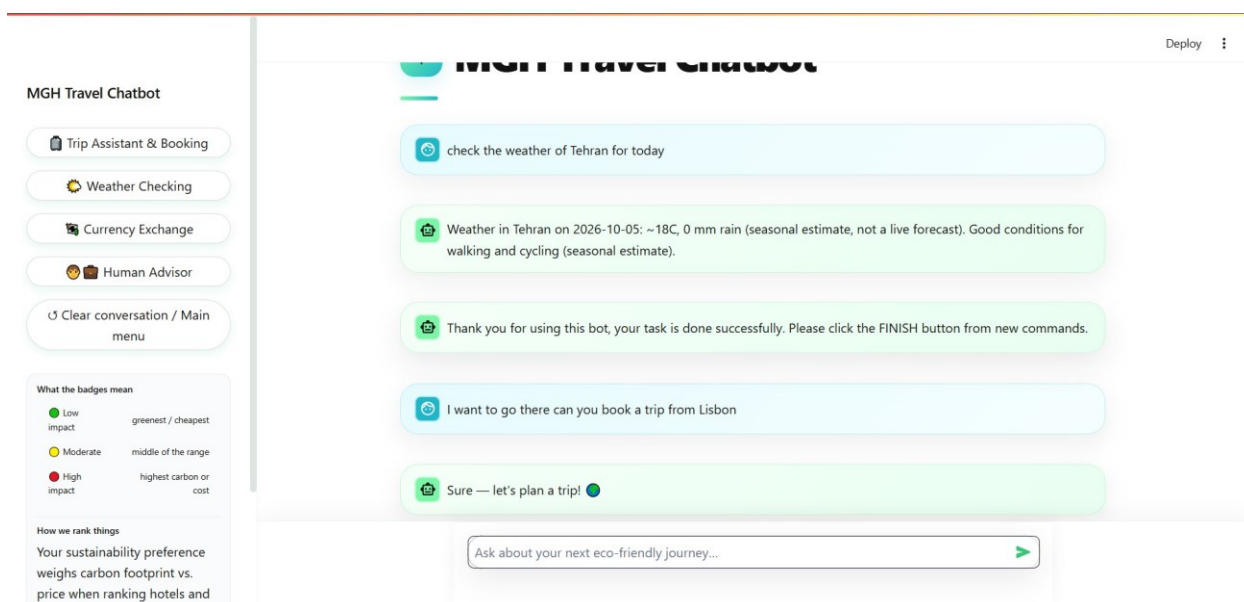


Figure 11

Task 3: Creation of Conversational Prototype and User Interaction Design

The prototype is the working chatbot, called MGH Travel Chatbot in the chat to make it more personalise. I have build it with Streamlit on the Rasa REST chanell. The Streamlit let me to build the result cards in Python, without JavaScript pretty easilly. I have drew the figures with Miro, so I could change them as the bot changed so simply aswell. My architecture document and the big picture of the Miro (that is attached to the github repo)architecture is showing the integration points and The Miro board shows the same flow with the integration points in a large scope. The next figure shows what replaced the tools named in the brief (Task 4 explains why it's like that). In [Miro link](#) you can easily find the full architecture in high resolution.

The Miro picture is showing the home screen architecture: four main flows (trip planning and booking, weather, currency, human advisor) and two helpers (hotel list and cities list). Every question accepts typing, and most also have buttons.

See the Figure 12 in the next page.

Scenario	What the bot does
Short city break, such as London to Paris	Show coach, train and flight, because the trip is less than 500 km.
Long trip, like Berlin to Tehran	Shows flights only, because train is offered up to 1,000 km and bus is up to 500 km.
Different sustainability levels	High adds a priority question and gives carbon 70% of the weight (85% for carbon priority). Medium gives 50% and low 25%.

The interface have all of the the elements the brief lists'. Quick-reply bottuns cover destinations, dates, travellers, budget ranges and yes or no bottuns also. Hotels are appearing as a short list in the trip card and as big list on the List of Hotel bottun. Each option has a green, amber or red badge with words, and the red badge warns about high-emission options. After a handover request, a card will show up a reference code for customer.

The all users can change the course at any step (Figure 11 as I attached above). After a weather check or hotel list, "I want to go there" (or some sentence like that) will start a trip to that city and skips the destination question, if you mentioned the origin city aswell the bot can understand it and dlsowill skip the origin_city question too. You can also change the destination or switch to another task, and Finish or Clear returns to the home screen. Meanwhile of a booking (until you didn't say yes to the question: Would you like to book this?) you can even change the destination also by saying : may I change my trip to Boston?

Users dislike chatbots that misunderstand them (Følstad and Brandtzaeg, 2020), so each question accepts two way: buttons and typing. A sidebar legend explains the badges and the ranking, because an explanation of an eco-label's purposes made air passengers preffer greener flights more strongly (Baumeister, Zeng and Hoffendahl, 2022).

Task 4: Programming Work, AI, and NLP Techniques

My bot have Rasa NLU and Core, a custom action file server also, one data service and also one Streamlit interface awsell, which the lecturer approved for this prototype instead of Rasa Webchat or ReactJS. NLU uses DIET with a whitespace tokeniser and count vector faetures, and Regex(re) extractor for bringing out the cities names and dates from text, that run next to it. DIET needs no large pre-trained embeddings despite of LLMs, so I did not add spaCy or the optional DistilBERT pipeline (Bunk et al., 2020).

The four main flows of my architecture document are using Rasa forms. They are asking the questions and also filling 26 slots, such as destination, dates and budget, and 71 rules route of the flows. My custom fallback nudges until the user give the answer of that question or clicl on the other function from left side of the bot, declines with a Clear button in the left, and hands over to a human by writing human advisor. It replaces action_default_fallback, which only send one generic reply and revert the chat to before the user's message. But anytime after the fallback message you answer the question and it will be saved in the right slot.

The actions call my data service, not public APIs. It tries the live source twice (OpenStreetMap, Open-Meteo, Frankfurter or Wikipedia), then will answers from saved data if they don't send any responds. Each action catches API errors and sends a plain message, so one failed call does not stop the trip card at all. To run the the whole apis you need to run this command in the folder of the bot "uvicorn data_services.main:app --port 8000"

Tool in the brief	What I used instead	Why
Climatiq (carbon)	A local table of emission factors per passenger-km	A trip needs one figure per mode. A table needs no key.
Amadeus (flights, hotels)	Flights: saved routes and a distance formula. Hotels: OpenStreetMap	Its self-service API keys were to be disabled on 17 July 2026 (Crotty, 2026).
OpenRouteService (routing)	Saved routes, otherwise the haversine formula	It was not provided.
OpenCage (geocoding)	About 170 saved cities, then Nominatim	Nominatim need no key. I built no GPS.
Mock JSON database (eco data)	Hotel eco tags from OpenStreetMap. Fixed text on two offset registries	I built no separate database.

The ranking function scales the carbon and price across the whole options of one trip, then add them with weights from the sustainability level (which has been expressed above). It also will rank transport and hotels. A rule hides coach beyond 500 km and train beyond 1,000 km and more than that flight comes in also. The badge shows which third of the ranking an option is in, and Streamlit builds the buttons, cards and badges from what the action sending. The handover action packs the slots, the last intent and the last 20 chat line, and gives a reference code.

After a weather check or hotel list, the intent ask_trip_to_this_city runs action_start_trip_from_weather_drift, which copies the last checked city into destination_city. Other

actions change the destination mid-form or switch will flow. A user can also fill four slots with one sentence at the same time .

The formula is: intent travel + from travel_date_from + to travel_date_to + from origin_city + to destination_city. (I want to go to travel + from today + to tomorrow + from Berlin + to Paris)

My city and date extractors reads it and count the nights with a inside system, so the bot skips questions 1 to 4 and asks how many people travel. Typing one such sentence is the fastest way to plan a trip. Table 5 shows three sentences I tested.

Sentence	Slots filled
I want to go to a travel from today to next week from Paris to Berlin	Berlin, Paris, 7 nights
I want to go to a travel from this weekend to next weekend from london to paris	Paris, London, 7 nights
I want to go to a travel from tomorrow to next week from Berlin to Tehran	Tehran, Berlin, 7 nights

Limits: straight-line distance makes train and coach carbon a little low. At the low and medium levels the badge follows price as much as carbon, so a cheap flight can be green and a train red. The budget will be shown but doesn't filter results. There is no voice input. The code still holds an unused ClimaTiq helper for further developments.

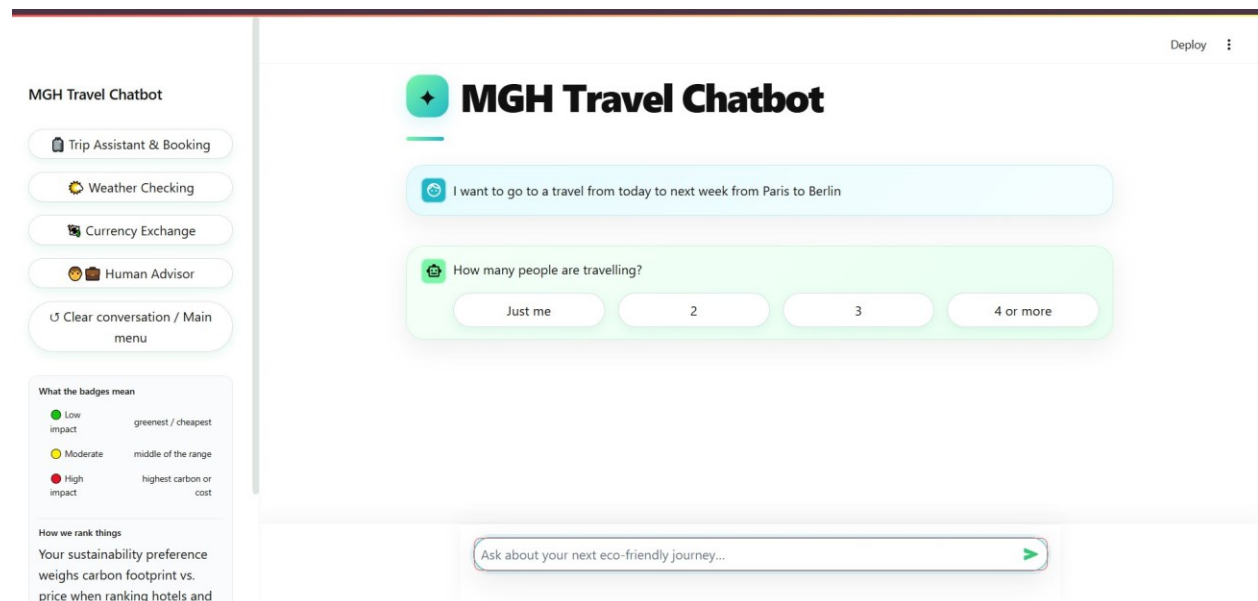


Figure 13

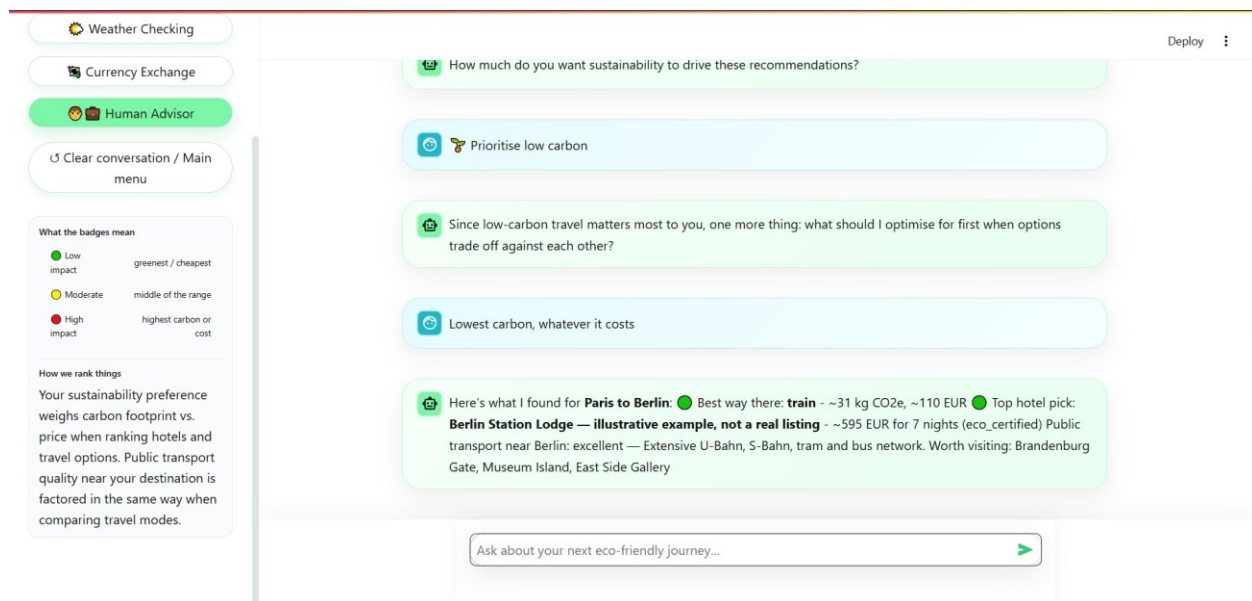


Figure 4

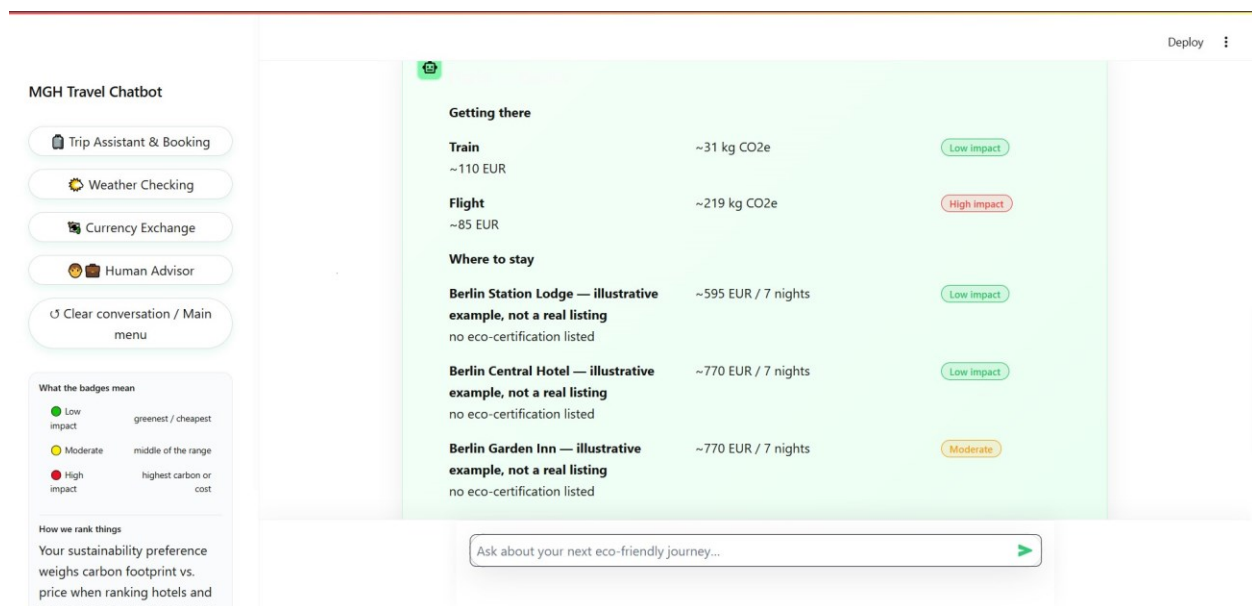


Figure 5

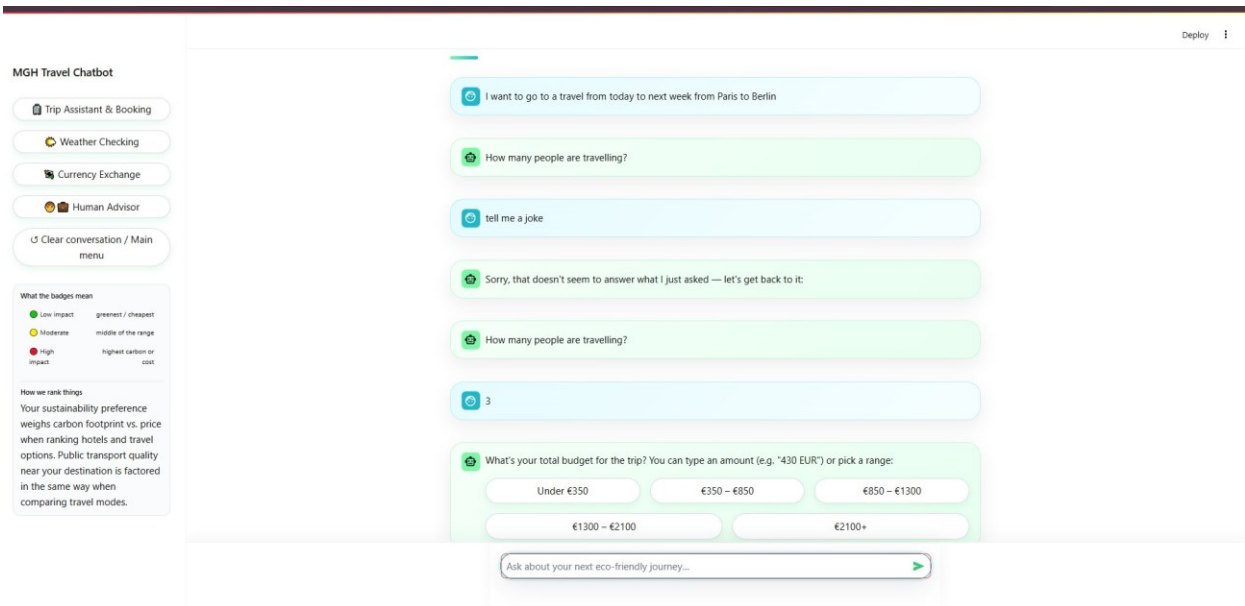


Figure 14

Task 5: Testing

I've tested my bot with Rasa's test commands and pytest and hand tests many times in addition to ten people. In the next table which is showing all of my runs from 6 October 2026. Rasa's story tests only is checking which actions runs, so I added pytest to check what the actions say actually(Rasa, n.d.c).

To run the pytest and even whole project in a correct way, you may need to install the requirements as explained in below:

First: `pip install -r requirements.txt` and `pip install -r requirements-dev.txt`

Then: `pytest tests/ -v > pytest_results.txt`

Test	What I did run	The Results
NLU, held-out	268 new phrases (trained on 520; 29 intents, 14 entities)	Accuracy 75.0%. Six intents reach F1 0.85.
NLU, 5-fold cross-validation	Each fold 80/20	Test accuracy 71.7%, train 99.7%. Entity F1 0.48.
Core	33 stories: happy paths, three sustainability levels, task switches, off-topic input	All passed. TEDPolicy decided nothing, so these test my rules. Validation finds no conflicts.
pytest	206 tests, with simulated outages, the duplicate form-run guard and a no-overlap check of the NLU sets	206 will pass. Just Two city-button tests fail.
Typed answers in forms	30 held-out answers typed at the matching questions	Budget 8 of 8, sustainability 6 of 7, priority 4 of 6, travellers 2 of 9.
Recommendation	6 routes, 3 levels	High: lowest carbon first. Low: cheapest first. All routes.

The target of F1 0.85 per intent isn't met. Short answers fail most: the five of eight budget phrases was read as out_of_scope slot, and cross-validation mixed priority focus with sustainability levels. From 67 mistakes, 48 fall below the 0.65 fallback thresholds, so the bot wants to ask again. And four runings with extra examples between them just stayed between 73.5% and 76.5%, so the data set needs to get grow. Forms read the typed text themselves and save most waek intents.

Hand tests passed for greetings, "are you a bot?", the typos like "brrlin" (the bot asks "Is this what you mean? Berlin" to get sure about the user meaning), budgets like "350-850", the Finish reminder, "No, not this time", and typed carbon footprint and "see more options" after booking. Unreadable dates or budgets will br counting as strikes. Three sentences from the table in former task fill four slots at once (I talked about it completely in task4), for example "I want to go to a travel from today to next week from Paris to Berlin". This phrase : "EUR350 - EUR850" will make the bot to start a currency exchange because of the name of the currency in it.

From a trip from Paris to Berlin at the low and medium levels, the flight (219 kg) have a green badge and the train (31 kg) a red one. With two live sources got blocked at my test machine, all 18 trips still coming back in about 1.3 second. Only places' names, coordinating and currency codes leave the bot, but there

is no privacy notice. Prices are kind of estimating, and obviously booking is a mock, and the offset text says offsets won't cancel a flight. This bot is working in English only, and has no voice or screen-reader test. Badges carry words as well as their colours (European Union, 2016; Lister et al., 2020; World Wide Web Consortium, 2024).

Ten people tested the bot. They said talking to bot with voice would be a better option, the looks matter and feel really good, switching between functions also is working well, and the whole chat should stay visible instead of clearance. Voice is future work. Finish and Clear still clear the chat.

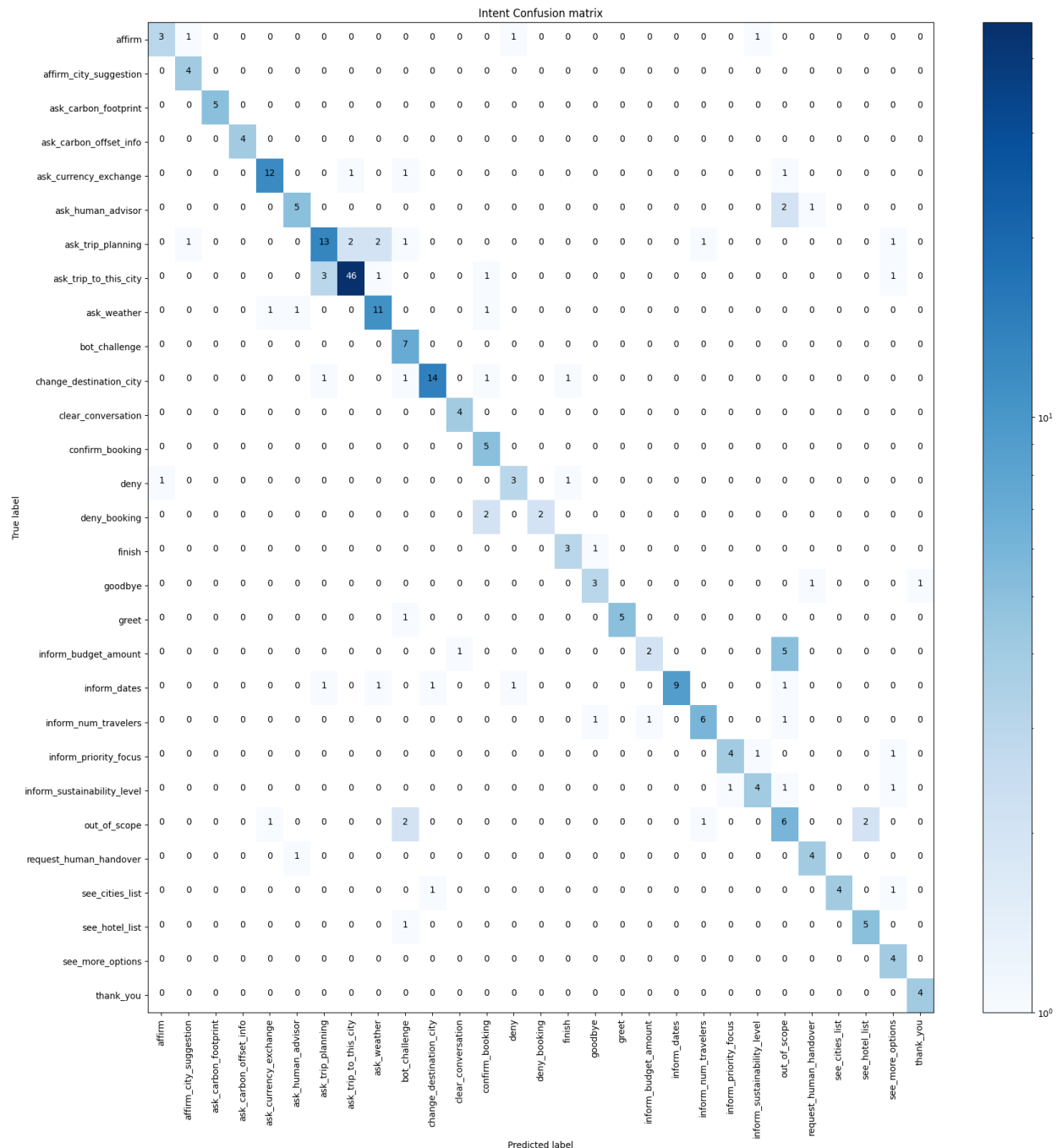


Figure 15

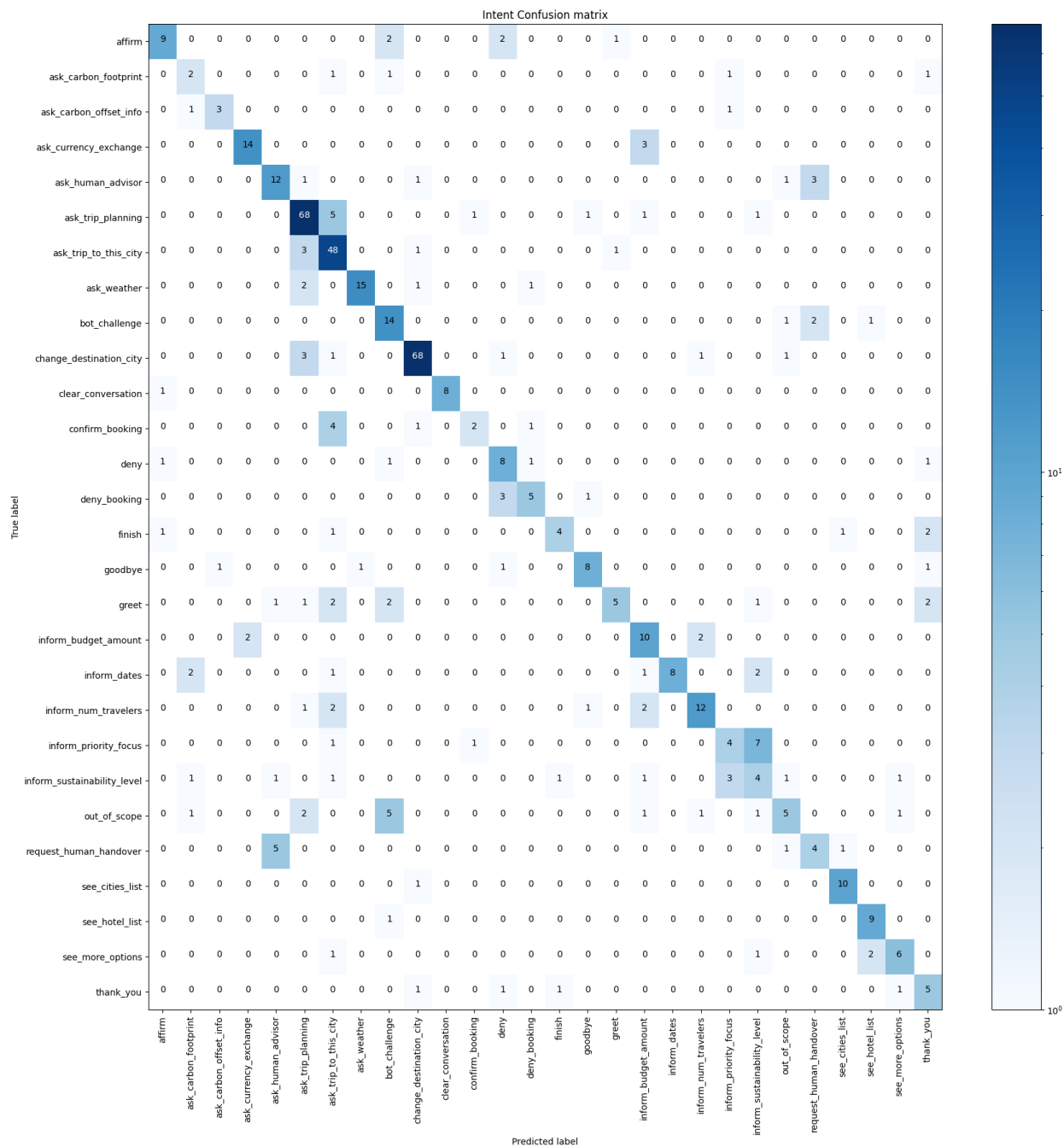


Figure 16

```

C:\Rasa\eco-travel-advisor>rasa test nlu
C:\Rasa\env\Lib\site-packages\rasa\core\tracker_store.py:1044: MovedIn20Warning: Deprecated API features detected! These feature(s) are not compatible with SQLAlchemy 2.0. To prevent in
compatible upgrades prior to updating applications, ensure requirements files are pinned to "sqlalchemy<2.0". Set environment variable SQLALCHEMY_WARN_20=1 to show all deprecation warnings.
Set environment variable SQLALCHEMY_SILENCE_UBER_WARNING=1 to silence this message. (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
Base: DeclarativeMeta = declarative_base()
C:\Rasa\env\Lib\site-packages\rasa\shared\utils\validation.py:134: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
import pkg_resources
C:\Rasa\env\Lib\site-packages\pkg_resources\__init__.py:3117: DeprecationWarning: Deprecated call to 'pkg_resources.declare_namespace('mpl_toolkits')'.
Implementing implicit namespace packages (as specified in PEP 420) is preferred to 'pkg_resources.declare_namespace'. See https://setuptools.pypa.io/en/latest/references/keywords.html#keywor
d-namespace-packages declare_namespace(pkg)
C:\Rasa\env\Lib\site-packages\pkg_resources\__init__.py:3117: DeprecationWarning: Deprecated call to 'pkg_resources.declare_namespace('ruamel')'.
Implementing implicit namespace packages (as specified in PEP 420) is preferred to 'pkg_resources.declare_namespace'. See https://setuptools.pypa.io/en/latest/references/keywords.html#keywor
d-namespace-packages declare_namespace(pkg)
C:\Rasa\env\Lib\site-packages\tensorflow\lite\python\util.py:52: DeprecationWarning: jax.xla_computation is deprecated. Please use the AOT APIs.
from jax import xla_computation as _xla_computation
C:\Rasa\env\Lib\site-packages\matplotlib\fontconfig_pattern.py:66: PyparsingDeprecationWarning: 'setParseAction' deprecated - use 'set_parse_action'
).setParseAction(self._family)
C:\Rasa\env\Lib\site-packages\matplotlib\fontconfig_pattern.py:127: PyparsingDeprecationWarning: 'parseString' deprecated - use 'parse_string'
self._parser.parseString(pattern)
C:\Rasa\env\Lib\site-packages\matplotlib\fontconfig_pattern.py:134: PyparsingDeprecationWarning: 'resetCache' deprecated - use 'reset_cache'
self._parser.resetCache()
2026-10-06 10:26:36 INFO rasa.core.processor - Loading model models\20261005-125700-jolly-bookcase.tar.gz...
C:\Rasa\env\Lib\site-packages\rasa\utils\train_utils.py:530: UserWarning: constrain_similarities is set to 'False'. It is recommended to set it to 'True' when using cross-entropy loss.
rasa.shared.utils.io.raise_warning(
2026-10-06 10:27:06 INFO rasa.nlu.test - Running model for predictions:
100% | 520/520 [00:04<00:00, 111.86it/s]
2026-10-06 10:27:11 INFO rasa.nlu.test - Intent evaluation results:
2026-10-06 10:27:11 INFO rasa.nlu.test - Intent Evaluation: Only considering those 520 examples that have a defined intent out of 520 examples.
2026-10-06 10:27:11 INFO rasa.nlu.test - Classification report saved to results\intent_report.json.
2026-10-06 10:27:11 INFO rasa.nlu.test - Incorrect intent predictions saved to results\intent_errors.json.
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:30: PyparsingDeprecationWarning: 'enablePackrat' deprecated - use 'enable_packrat'
ParserElement.enablePackrat()
2026-10-06 10:27:12 INFO rasa.utils.plotting - Confusion matrix, without normalization:

```

Figure 17

```

C:\Rasa\env\Lib\site-packages\matplotlib\fontconfig_pattern.py:134: PyparsingDeprecationWarning: 'resetCache' deprecated - use 'reset_cache'
self._parser.resetCache()
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:2022: PyparsingDeprecationWarning: 'setName' deprecated - use 'set_name'
val.setName(key)
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:2033: PyparsingDeprecationWarning: 'oneOf' deprecated - use 'one_of'
p.space <= oneOf(list(self._space_widths))
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:1878: PyparsingDeprecationWarning: 'setParseAction' deprecated - use 'set_parse_action'
empty.setParseAction(raise_error)
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:2047: PyparsingDeprecationWarning: 'leaveWhitespace' deprecated - use 'leave_whitespace'
+ SuppressRegex("(?![A-Za-z])").leaveWhitespace()
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:2289: PyparsingDeprecationWarning: 'unquoteResults' argument is deprecated, use 'unquote_results'
p.math_string <= QuotedString('$', '\\', unquoteResults=False)
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:2237: PyparsingDeprecationWarning: 'parseString' deprecated - use 'parse_string'
result = self._expression.parseString(s)
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:2245: PyparsingDeprecationWarning: 'resetCache' deprecated - use 'reset_cache'
self._expression.resetCache()
2026-10-06 10:27:14 INFO rasa.nlu.test - Entity evaluation results:
2026-10-06 10:27:14 INFO rasa.nlu.test - Evaluation for entity extractor: DIETClassifier
2026-10-06 10:27:14 INFO rasa.nlu.test - Classification report saved to results\DIETClassifier_report.json.
2026-10-06 10:27:14 INFO rasa.nlu.test - Incorrect entity predictions saved to results\DIETClassifier_errors.json.
2026-10-06 10:27:14 INFO rasa.utils.plotting - Confusion matrix, without normalization:
[[ 3  0  0  0  0  0  0  0  0  0  0]
 [ 0 11  0  0  0  0  1  0  0  0  0]
 [ 0  0  9  0  0  0  0  0  0  0  0]
 [ 0  0  0 120  0  0  0  0  0  0  0]
 [ 0  0  0  0  6  0  0  0  0  0  0]
 [ 0  0  0  1  0 2701  0  0  0  0  0]
 [ 0  0  0  0  0  0  8  0  0  0  0]
 [ 0  0  0  0  0  0  0 60  0  0  0]
 [ 0  0  0  0  0  0  0  0 65  0  0]
 [ 0  0  0  0  0  0  0  0  0 91  0]
 [ 0  0  0  0  0  0  0  0  0  0 6]]
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:2237: PyparsingDeprecationWarning: 'parseString' deprecated - use 'parse_string'
result = self._expression.parseString(s)
C:\Rasa\env\Lib\site-packages\matplotlib\matplotlib.py:2245: PyparsingDeprecationWarning: 'resetCache' deprecated - use 'reset_cache'
self._expression.resetCache()
(rasa-env) C:\Rasa\eco-travel-advisor>

```

Figure 18

```

===== warnings summary =====
..\Rasa\env\Lib\site-packages\starlette\formparsers.py:10
C:\Rasa\env\Lib\site-packages\starlette\formparsers.py:10: PendingDeprecationWarning: Please use 'import python_multipart' instead.
import multipart
..\Rasa\env\Lib\site-packages\httpx_client.py:690
C:\Rasa\env\Lib\site-packages\httpx_client.py:690: DeprecationWarning: The 'app' shortcut is now deprecated. Use the explicit style 'transport=WSGITransport(app=...)' instead.
warnings.warn(message, DeprecationWarning)
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 206 passed, 2 warnings in 119.16s (0:01:59) =====
(rasa-env) C:\Rasa\eco-travel-advisor>

```

Figure 19

Task 6: Deployment

I ran the bot as four processes: the data service (port 8000), action server (5055), and the Rasa server (5005) and the Streamlit on 8501. My README is listing the steps also. I create a Python 3.10 virtual environment and install requirements.txt, where every version is pinned because Rasa 3.6.21 doesn't run on Python 3.11. Then I run rasa data validating and rasa train, and started the four processes. The requirements-dev.txt adds pytest for the running the tests. The trained model and the test results are in .gitignore, so everyone who clones the repo have to trains the model at first. The code is on GitHub: [Link](#).

This repo also have some Dockerfiles which was related to the dockerizing the project for huggingFace. The Dockerfile will build one image that validates the data and trains the model during the build. The compose file will start the four processes in four containers. I did not deploy them, so they are unused aswell. Also no need to use AWS, Azure or GCP either, and the brief's diagram warns of billing risk on AWS and GCP. Hugging Face Spaces, the brief's recommendation, now needs a paid plan for Docker Spaces (Hugging Face, n.d.).

No API key is needed also , so the repository holds no secrets. The codes reads three variables with local defaults: RASA_URL (Streamlit), DATA_SERVICES_URL (action server) and the PORT (container). CLIMATIQ_API_KEY is also read but has no effect here. The .env file is in .gitignore and .env.example is kept. In the single container only Streamlit's port is exposed. The compose file opens all four ports, so it is for local use only. Rasa runs with CORS open to all and no token, so I would add a token or a firewall rule before any public use. Rasa advises not to expose its server directly and to secure it with a firewall or an authentication method (Rasa, n.d.d).

Part	What it does
Public transport	Stop counts from OpenStreetMap, or a saved rating for about 20 cities. No timetables.
Train and coach fares	Estimates, not quotes: coach 8 EUR + 0.045 EUR per km, train 12 EUR + 0.11 EUR per km.
Cards	Streamlit draws trip summary, more hotels and handover cards from one registry.
Reset	A reset_to_menu marker makes Streamlit clear the chat, like the Clear button.
Session	A session ends after 60 minutes and the slots carry over.

Conclusion

I built a chatbot that plans a trip and ranks the options by carbon, price and the user's preference. It uses Rasa with DIET, forms and rules, one data service with saved data as a backup, and a Streamlit page. It also checks the weather, converts currency and hands the chat to a human advisor.

All 33 Core stories pass, and 204 of 206 pytest tests pass. The NLU reaches 75.0% accuracy on new phrases, but only six of 29 intents reach F1 0.85, so my target is not met. I also found faults that I did not fix.

Ten users liked the look and the easy switching between functions. They asked for voice and for a chat that stays visible. Next I would add training examples, fix the faults, add voice and a privacy notice, and deploy the bot. Booking stays a mock, and prices and fares stay estimates.

References

- Baumeister, S., Zeng, C. and Hoffendahl, A. (2022) 'The effect of an eco-label on the booking decisions of air passengers', *Transport Policy*, 124, pp. 175-182. doi:10.1016/j.tranpol.2020.07.009.
- Bunk, T., Varshneya, D., Vlasov, V. and Nichol, A. (2020) DIET: lightweight language understanding for dialogue systems. arXiv:2004.09936. Available at: <https://arxiv.org/abs/2004.09936> (Accessed: 5 October 2026).
- Crotty, A. (2026) 'Amadeus to shut down self-service APIs portal for developers', *PhocusWire*, 9 February. Available at: <https://www.phocuswire.com/amadeus-shut-down-self-service-apis-portal-developers> (Accessed: 6 October 2026).
- European Union (2016) Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation). *Official Journal of the European Union*, L 119, pp. 1-88. Available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (Accessed: 5 October 2026).
- Følstad, A. and Brandtzaeg, P.B. (2020) 'Users' experiences with chatbots: findings from a questionnaire study', *Quality and User Experience*, 5, article 3. doi:10.1007/s41233-020-00033-2.
- Gössling, S. and Humpe, A. (2020) 'The global scale, distribution and growth of aviation: implications for climate change', *Global Environmental Change*, 65, article 102194. doi:10.1016/j.gloenvcha.2020.102194.
- Hugging Face (n.d.) Spaces overview. Available at: <https://huggingface.co/docs/hub/spaces-overview> (Accessed: 6 October 2026).
- Lister, K., Coughlan, T., Iniesto, F., Freear, N. and Devine, P. (2020) 'Accessible conversational user interfaces: considerations for design', *Web for All Conference (W4A 2020)*, Taipei, 20-21 April. Available at: <https://oro.open.ac.uk/69720/> (Accessed: 5 October 2026).
- Majid, G.M., Tussyadiah, I. and Kim, Y.R. (2025) 'Exploring the potential of chatbots in extending tourists' sustainable travel practices', *Journal of Travel Research*, 64(6). doi:10.1177/00472875241247316.
- Rasa (n.d.a) Tracker stores. Available at: <https://legacy-docs-oss.rasa.com/docs/rasa/tracker-stores> (Accessed: 5 October 2026).
- Rasa (n.d.b) Fallback and human handoff. Available at: <https://legacy-docs-oss.rasa.com/docs/rasa/fallback-handoff> (Accessed: 6 October 2026).
- Rasa (n.d.c) Testing your assistant. Available at: <https://legacy-docs-oss.rasa.com/docs/rasa/testing-your-assistant> (Accessed: 6 October 2026).

Rasa (n.d.d) Rasa HTTP API. Available at: <https://legacy-docs-oss.rasa.com/docs/rasa/http-api> (Accessed: 6 October 2026).

Sun, Y.-Y., Faturay, F., Lenzen, M., Gössling, S. and Higham, J. (2024) 'Drivers of global tourism carbon emissions', *Nature Communications*, 15. doi:10.1038/s41467-024-54582-7.

Travalyst (2024) Flight emissions data shown in over 65 billion searches worldwide. 12 September. Available at: <https://travalyst.org/news/flight-emissions-data-shown-in-over-65-billion-searches-worldwide/> (Accessed: 5 October 2026).

Triki, A. (2026) Eco-Travel Advisor: conversational agent for sustainable tourism planning using the Rasa platform. Assignment brief. Advanced Conversational UI Design and Chatbot Development, MSc Artificial Intelligence. Berlin School of Business and Innovation.

World Wide Web Consortium (2024) Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 12 December. Available at: <https://www.w3.org/TR/WCAG22/> (Accessed: 5 October 2026).

Xie, J., Zhang, K., Chen, J., Zhu, T., Lou, R., Tian, Y., Xiao, Y. and Su, Y. (2024) 'TravelPlanner: a benchmark for real-world planning with language agents', *Proceedings of the International Conference on Machine Learning (ICML)*, PMLR 235, pp. 54590-54613.

APPENDIX



Github repo : [Link](#)

Miro : [Link](#)